

Competitive Programming

Section 18: DSU And Minimum Spanning Trees

Competitive Programming Course

2026-07-05

Table of contents

1	Why DSU	3
2	Recognition	4
3	DSU vocabulary	5
4	The find operation	6
5	The union operation	7
6	Worked example: edge additions	8
7	Minimum spanning tree	9
8	Kruskal's algorithm	10
9	Why Kruskal is correct	11
10	Worked example: Kruskal	12
11	Clustering by stopping Kruskal	13
12	Complexity	14
13	Implementation notes	15
13.1	C++	15
13.2	Python	15
14	Local exercises	16
14.1	A. Network Growth	16
14.2	B. Minimum Network	16
14.3	C. Cluster Split	16
15	Submission checklist	17

1 Why DSU

Many graph problems do not need shortest paths or traversal order. They only ask:

```
Are these two vertices already connected?  
If I add this edge, do two components merge?  
How many components remain?  
What is the cheapest way to connect everything?
```

Disjoint set union, usually called DSU or union-find, is built for this.

DSU maintains a partition of vertices into connected components while edges are added. It supports two operations:

```
find(x)      -> representative of x's current component  
union(a,b)   -> merge the two components if they are different
```

2 Recognition

Look for these signals:

Statement signal	Likely tool
edges are added, never deleted	DSU
ask whether two vertices are connected after updates	DSU
maintain number or size of components	DSU
choose cheapest edges connecting all vertices	Kruskal + DSU
split graph into k clusters using cheap edges	Kruskal variant

If edges are deleted, ordinary DSU is not enough. That requires later offline or rollback techniques.

3 DSU vocabulary

Each component has one representative, also called its root.

Example components:

```
{1,2,5}, {3}, {4,6}
```

The representative is just an internal label. It does not need to be the smallest vertex.

In arrays:

```
parent[x] = parent pointer of x  
size[root] = number of vertices in root's component
```

Initially every vertex is alone:

```
parent[x] = x  
size[x] = 1
```

4 The find operation

find(x) follows parent pointers until it reaches a root:

```
x -> parent[x] -> parent[parent[x]] -> ... -> root
```

Path compression makes future calls faster by pointing visited vertices closer to the root.

Python:

```
def find(x):  
    while parent[x] != x:  
        parent[x] = parent[parent[x]]  
        x = parent[x]  
    return x
```

C++:

```
int find(int x) {  
    while (parent[x] != x) {  
        parent[x] = parent[parent[x]];  
        x = parent[x];  
    }  
    return x;  
}
```

5 The union operation

To merge components:

```
ra = find(a)
rb = find(b)
if ra == rb: already connected
otherwise attach smaller component under larger component
```

Union by size:

```
if size[ra] < size[rb], swap ra and rb
parent[rb] = ra
size[ra] += size[rb]
```

This keeps trees shallow. Combined with path compression, DSU operations are effectively constant time for contest purposes.

6 Worked example: edge additions

Start with five vertices:

```
{1} {2} {3} {4} {5}
components = 5
largest = 1
```

Add edge 1-2:

```
{1,2} {3} {4} {5}
components = 4
largest = 2
```

Add edge 3-4:

```
{1,2} {3,4} {5}
components = 3
largest = 2
```

Add edge 2-3:

```
{1,2,3,4} {5}
components = 2
largest = 4
```

Add edge 1-4:

```
1 and 4 already have the same representative.
No component changes.
```

7 Minimum spanning tree

For an undirected weighted graph, a spanning tree is a set of edges that:

```
connects all vertices  
uses exactly  $n-1$  edges  
has no cycle
```

A minimum spanning tree, or MST, is a spanning tree with minimum total weight.

If the graph is disconnected, no spanning tree exists.

8 Kruskal's algorithm

Kruskal's algorithm is the standard DSU-based MST algorithm:

```
sort all edges by weight
start with no chosen edges
for each edge in increasing weight:
    if its endpoints are in different components:
        choose the edge
        union the components
    otherwise skip it
```

DSU answers the key question:

```
Would adding this edge create a cycle?
```

If both endpoints are already in the same component, adding the edge creates a cycle. If they are in different components, adding the edge safely merges two parts.

9 Why Kruskal is correct

The core idea is the cut property.

Take any split of vertices into two sides:

`S and everything not in S`

An edge crossing the split is called a cut edge.

The cut property says:

`The lightest edge crossing any cut is safe to include in some MST.`

Kruskal repeatedly chooses the lightest edge that connects two different current components. That edge is the lightest edge crossing the cut between one component and the rest, so it is safe.

10 Worked example: Kruskal

Edges:

```
1-3 weight 1
2-3 weight 2
1-2 weight 3
2-4 weight 4
3-4 weight 5
```

Sorted order:

```
(1-3,1), (2-3,2), (1-2,3), (2-4,4), (3-4,5)
```

Process:

```
take 1-3, cost 1
take 2-3, cost 3
skip 1-2 because 1 and 2 are already connected
take 2-4, cost 7
```

We used $n-1 = 3$ edges, so the MST cost is 7.

11 Clustering by stopping Kruskal

Kruskal also solves a useful clustering variant:

```
connect vertices as cheaply as possible until exactly k components remain
```

Start with n components. Each successful union reduces the component count by one. Stop when the count is k .

This is equivalent to taking the cheapest safe edges that do not reduce the number of groups below k .

12 Complexity

Sorting dominates:

```
Kruskal time:  $O(m \log m)$   
DSU memory:  $O(n)$   
Graph memory:  $O(m)$ 
```

For pure edge additions without sorting:

```
Time: almost  $O(m)$   
Memory:  $O(n)$ 
```

13 Implementation notes

13.1 C++

Use `long long` for MST costs:

```
long long total = 0;
```

Store weighted edges as tuples:

```
vector<tuple<long long,int,int>> edges;  
sort(edges.begin(), edges.end());
```

13.2 Python

Python tuples sort lexicographically, so this works:

```
edges.append((w, u, v))  
edges.sort()
```

Use iterative `find`; recursive `find` is short, but iterative avoids recursion concerns.

14 Local exercises

14.1 A. Network Growth

Maintain:

```
components  
largest component size
```

after each added edge.

14.2 B. Minimum Network

Run full Kruskal. If fewer than $n-1$ edges are chosen, print IMPOSSIBLE.

14.3 C. Cluster Split

Run Kruskal until exactly k components remain. If the available edges cannot merge enough components, print IMPOSSIBLE.

15 Submission checklist

Before submitting:

```
Did I convert vertices to 0-indexed arrays?  
Did find() return the representative after path compression?  
Did union() return false when endpoints were already connected?  
Did I sort edges by weight before Kruskal?  
Did I use long long for total cost in C++?  
Did I check disconnected/impossible cases?
```